CSA0313 • DATA STRUCTURES

Recursion, the Call Stack & Space Complexity of Factorial

How a simple recursive function quietly builds — and unwinds — a stack

PRESENTED BY

Tejasri M.

Reg No: 192511047

GUIDED BY

Dr. M. Shakila

FOUNDATIONS

What Is Recursive Factorial?

A function that solves n! by calling a smaller version of itself

factorial.py

```
def factorial(n):
    if n == 0:          # base case
        return 1
    return n * factorial(n - 1)

factorial(4)
# -> 4 * factorial(3)
# -> 4 * (3 * factorial(2))
# -> 4 * (3 * (2 * factorial(1)))
# -> 4 * (3 * (2 * (1 * factorial(0))))
# -> 24
```

B

Base Case

n == 0 returns 1 immediately, with no further call. Without it, recursion never terminates.

R

Recursive Case

Each call reduces the problem to factorial(n-1) and waits for that result before multiplying by n.

INTERNAL MECHANICS

Recursion Runs on the Call Stack

Every call to factorial(n) pushes a new stack frame — LIFO order

Each stack frame stores:

- The parameter n for that call
- A return address — where to resume the caller
- Local variables and the pending multiplication (n × …)
- Saved register/context values for that invocation

Frames are only popped once the base case returns — the multiplications happen during the unwind, not the descent.

TRACE

Two Phases: Descent (Push) & Unwind (Pop)

PUSH ↓ (calls made)	POP ↑ (results returned)
`call factorial(4)`	`factorial(0) returns 1`
`call factorial(3)`	`factorial(1) returns 1×1 = 1`
`call factorial(2)`	`factorial(2) returns 2×1 = 2`
`call factorial(1)`	`factorial(3) returns 3×2 = 6`
`call factorial(0) → hits base`	`factorial(4) returns 4×6 = 24`

Peak stack depth = 5 frames (n+1) — reached the instant the base case is hit, before any pop occurs.

ANALYSIS

Space Complexity: O(n) Call Stack

O(n)

space complexity

n + 1

max stack frames alive at once

O(1)

extra space per frame (n, return addr.)

Recursive vs. Iterative Factorial

Aspect	Recursive	Iterative
Data structure used	Implicit call stack	None (single variable)
Space complexity	O(n)	O(1)
Time complexity	O(n)	O(n)
Risk for large n	Stack overflow	No overflow risk
Readability	Mirrors math definition	More verbose

Each additional call adds a fixed-size frame, so total space scales linearly with input n.

CONCLUSION

Key Takeaways

1

Stack is implicit

Recursion doesn't use a data structure you write — the language runtime maintains a call stack of frames automatically.

2

LIFO discipline

Frames are pushed on each recursive call and popped only after the base case resolves, in strict last-in-first-out order.

3

Linear space cost

factorial(n) needs O(n) extra space for n+1 stack frames — a real cost absent from the iterative version.

4

Design trade-off

Choose recursion for clarity on bounded n; prefer iteration (or tail-call-optimized languages) when n or memory is a concern.